

Outlines

- Introduction
- Scope in C++
- Inline Function
- Reference Variables
- Pointer vs References
- Function Overloading
- Comparison between C and C++

Introduction to C++

- C++ is a high-level, general-purpose programming language
- C++ was developed by Bjarne Stroustrup at AT & T Bell lab, USA in early eighties.
- Supports multiple programming paradigms including procedural, object-oriented, and generic programming.
- C++ is a cross-platform language that can be used to create high-performance applications.
- C++ gives programmers a high level of control over system resources and memory.

Why C++?

- **Performance:** Optimal use of system resources and faster execution.
- **Control:** Greater control over system resources and memory management.
- **Versatility:** Suitable for a wide range of applications from gaming to real-time systems.
- **Community and Ecosystem:** Strong community support and extensive libraries and frameworks.
- **Industry Demand:** Widely used in industries such as game development, finance, and system software.

A Sample Program

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {  
    cout<<"Hello World!!!";  
    return 0;  
}
```

Scope in C++

Definition: Scope refers to the visibility and lifetime of variables and functions in different parts of the program.

Scope of a variable is the area of the program where the variable is valid.

Types:

- Local scope
- Global scope
- Namespace scope
- Class scope

Local Scope

Local Scope: Variables declared inside a function or block.

A local variable's scope is limited to the block where it is declared and cannot be accessed (set or read) outside that block.

They can only be accessed within that function or block.

They are created when the block or function is entered and destroyed when it is exited.

```
#include <iostream>
using namespace std;

void myFunction() {
    int localVar = 10; // Local scope
    cout << "Local Variable: " << localVar <<
endl;
}

int main() {
    myFunction();
    cout << localVar; // Generates Error:
localVar is not in scope
    return 0;
}
```

Global Scope

Global Scope: Variables declared outside of all functions.

A global variable's scope is the entire program, meaning it can be accessed from any function or block within the program.

They are created when the program starts and destroyed when the program ends.

```
#include <iostream>
```

```
using namespace std;
```

```
int globalVar = 20; // Global scope
```

```
void display() {
```

```
    cout << "Global Variable: " << globalVar <<  
endl;
```

```
}
```

```
int main() {
```

```
    display(); // Accessing globalVar  
    cout << "Global Variable in main: " <<  
globalVar << endl;  
    return 0;  
}
```

Namespace Scope

Namespace Scope: Variables and functions declared within a namespace.

The scope of these variables and functions is limited to the namespace in which they are declared.

They can be accessed using the namespace name (with scope resolution operator `::`) or by using the `'using'` directive.

```
#include <iostream>

using namespace std;

namespace MyNamespace {
    int nsVar = 30; // Namespace scope
    void display() {
        cout << "Namespace Variable: " << nsVar <<
endl;
    }
}

int main() {
    MyNamespace::display(); // Accessing using
namespace name
    using namespace MyNamespace;
    cout << "Namespace Variable in main: " << nsVar
<< endl; // Accessing after using directive
    return 0;
}
```

Class Scope

Class Scope: Member variables and functions of a class.

The scope of these variables and functions is the entire class.

They can be accessed within the class methods and on instances of the class.

```
#include <iostream>
using namespace std;
```

```
class MyClass {
public:
    int memberVar; // Class scope
    void display() {
        cout << "Member Variable: " << memberVar <<
endl;
    }
};

int main() {
    MyClass obj;
    obj.memberVar = 40;
    obj.display(); // Accessing class member
    return 0;
}
```

Inline Function

Definition: An inline function is a function that is expanded in line at the point of each call, rather than being invoked with a traditional function call mechanism.

Purpose: To reduce the overhead of function calls, especially for small, frequently called functions.

Syntax:

```
inline returnType functionName(parameters) {  
    // function body  
}
```

```
#include <iostream>
using namespace std;

inline int add(int a, int b) {
    return a + b;
}

int main() {
    int result = add(3, 5); // Inline function
    call
    cout << "Result: " << result << endl;
    return 0;
}
```

```
// Inline expansion (pseudo-code)
int main() {
    int result = 3 + 5; // Directly replaced by the function
    code
    cout << "Result: " << result << endl;
    return 0;
}
```

Direct Substitution: The function code is directly substituted in place of the function call.

Inline Functions

Benefits of Using Inline Functions

- **Performance Improvement:** Reduces function call overhead.
- **Code Expansion:** The function code is expanded at each call site.
- **Best Use Case:** Suitable for small, frequently called functions.

When Not to Use Inline Functions

- **Complex Functions:** Large functions may lead to code bloat.
- **Recursion:** Inline functions should not be recursive.
- **Compiler Discretion:** The compiler may ignore the inline request.

Reference Variables

Definition: A reference variable is an alias for another variable. It is a way to give another name to an existing variable.

Purpose: To provide a simple way to access and modify the value of a variable without copying it.

Syntax:

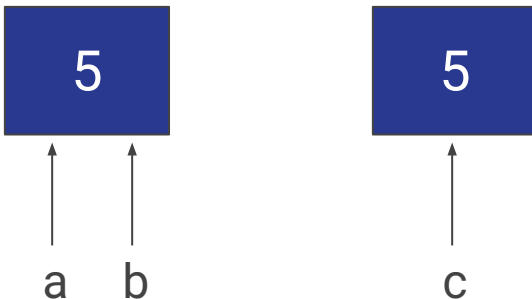
```
dataType &referenceName = existingVariable;
```

Example:

```
int a = 5;
```

```
int &b = a;
```

```
int c = a;
```



Reference Variables

How Do Reference Variables Work?

Alias: A reference variable is another name for an existing variable.

Direct Access: Any changes made to the reference variable affect the original variable directly.

Why Use Reference Variables?

Efficiency: Avoids the overhead of copying data when passing variables to functions.

Consistency: Changes to the reference reflect immediately on the original variable.

```
#include <iostream>
using namespace std;

int main() {
    int a = 10;
    int &ref = a; //ref is a reference to a
    ref = 20; // changes the value of a to 20
    cout << "a: " << a << endl; // Outputs: a:
    20
    return 0;
}
```

Reference Variables in Functions

Passing by Reference: Allows functions to modify the original arguments.

Syntax:

```
return_type functionName(dataType  
&param) {  
    // function body  
}
```

```
#include <iostream>  
using namespace std;  
  
void increment(int &num) {  
    num++;  
}  
  
int main() {  
    int value = 5;  
    increment(value); // value is now 6  
    cout << "value: " << value << endl;  
    // Outputs: value: 6  
    return 0;  
}
```

Reference Variables with const

Definition: A reference that cannot be used to modify the original variable.

Purpose: Useful for passing large objects to functions without copying while ensuring the function does not modify them.

```
#include <iostream>
using namespace std;

void printValue(const int &num) {
    cout << "Value: " << num << endl;
}

int main() {
    int value = 10;
    printValue(value); // Safe, the
                        // function cannot modify value
    return 0;
}
```

Example: Swap two numbers

```
#include <iostream>
using namespace std;

void swap(int &x, int &y) {
    int temp = x;
    x = y;
    y = temp;
}

int main() {
    int a = 5, b = 10;
    cout << "Before swap: a = " << a << ", b = " << b << endl;
    swap(a, b); // Swap values using references
    cout << "After swap: a = " << a << ", b = " << b << endl;
    return 0;
}
```

Pointer vs References

Pointer

What is a Pointer?

A pointer is a variable that stores the memory address of another variable.

Key Characteristics:

1. **Declaration:** Pointers are declared using the `*` operator.
2. **Nullability:** Pointers can be null (i.e., they can point to no object).
3. **Reassignable:** Pointers can be reassigned to point to different objects at any time.
4. **Arithmetic:** Pointers support pointer arithmetic (e.g., incrementing a pointer to point to the next memory location).

Example

```
#include <iostream>
using namespace std;

int main() {
    int var = 10;
    int* ptr = &var; // Pointer to var
    cout << "Value of var: " << var << endl;
    cout << "Address of var: " << &var << endl;
    cout << "Pointer ptr points to address: " << ptr << endl;
    cout << "Value at address ptr points to: " << *ptr << endl;

    int anotherVar = 20;
    ptr = &anotherVar; // Reassigning the pointer
    cout << "Pointer ptr now points to address: " << ptr << endl;
    cout << "Value at new address ptr points to: " << *ptr << endl;

    return 0;}
```

Output:

Value of var: 10

Address of var: 0x7ffdb76f9b88

Pointer ptr points to address: 0x7ffdb76f9b88

Value at address ptr points to: 10

Pointer ptr now points to address:
0x7ffdb76f9b8c

Value at new address ptr points to: 20

References

What is a Reference?

A reference is an alias for another variable. Once a reference is initialized to a variable, it cannot be changed to refer to another variable.

Key Characteristics:

1. **Declaration:** References are declared using the **&** operator.
2. **Non-nullability:** References must be initialized when declared and cannot be null.
3. **Non-reassignable:** References cannot be reassigned to refer to a different variable after initialization.
4. **Syntactic Simplicity:** References provide a way to access variables in a manner similar to pointers but with simpler syntax (no need to dereference).

Example

```
#include <iostream>
using namespace std;
int main() {
    int var = 10;
    int& ref = var; // Reference to var
    cout << "Value of var: " << var << endl;
    cout << "Reference ref refers to value: " << ref << endl;

    ref = 20; // Changing value via reference
    cout << "New value of var: " << var << endl;

    // References cannot be reassigned:
    int anotherVar = 30;
    ref = anotherVar; // It assigns the value of anotherVar to var (i.e. var = 30), Not reassign
ref to anotherVar
    return 0;}

```

Value of var: 10

Reference ref refers to value: 10

New value of var: 20

Comparison

Feature	Pointers	References
Declaration	<code>int* ptr = &var;</code>	<code>int& ref = var;</code>
Nullability	Can be null (e.g., <code>ptr = nullptr;</code>)	Cannot be null
Reassignable	Can be reassigned to point elsewhere	Cannot be reassigned
Initialization	Can be uninitialized initially	Must be initialized upon declaration
Syntax	Requires dereferencing (<code>*ptr</code>)	Accessed directly (<code>ref</code>)

Function Overloading

Function overloading in C++ allows programmer to define multiple functions with the same name but with different parameter lists.

This enables you to use the same function name for different purposes depending on the types or number of arguments passed to it.

Function overloading is particularly useful when you want to provide flexibility in the usage of your functions without having to come up with distinct names for similar operations.

It's a way to make your code more readable and expressive.

Function declarations that differ only in the return type cannot be overloaded. However, if the overloaded functions have varying data type or number of parameters, the return type can be different.

```
#include <iostream>
using namespace std;
// Function to add two integers
int add(int a, int b) {
    return a + b;
}
// Function to add two floating point numbers
float add(float a, float b) {
    return a + b;
}
// Function to add three int numbers
int add(int a, int b, int c) {
    return a + b + c;
}
int main() {
    int sum_int = add(3, 4); // Calls the first add function
    float sum_float = add(3.5f, 4.2f); // Calls the second add function
    int sum_int_three = add(2,3,4);

    cout << "Sum of integers: " << sum_int << endl;
    cout << "Sum of floats: " << sum_float << endl;
    cout << "Sum of 3 integers: " << sum_int_three << endl;
    return 0; }
```

C (Structured)	C++ (OOP)
C is a Procedure-oriented programming language	C++ is an Object-oriented programming language
Top down approach is used in program design	Bottom up approach is used in program design
More process or method-focused.	More data-focused, but also supports process-focused programming.
Does not support OOP features like polymorphism, encapsulation, and inheritance.	Supports OOP features like polymorphism, encapsulation, and inheritance.
Does not support reference variables.	Support reference variables.
Does not support function or operator overloading.	Supports both function and operator overloading.
Good for embedded devices, system-level code etc.	Good for developing games, networking, server-side applications etc.



Any Queries?